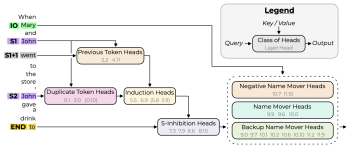
Deep Learning for NLP

Explainability and Interpretability

Explainability and Interpretability

Learning goals

- Distinguish feature attribution, training-data attribution, and mechanistic interpretability
- Explain how LIME, SHAP, and related methods attribute model outputs to input features
- Understand how training examples and internal model components can be linked to model behavior
- Distinguish plausible from faithful explanations and describe how explanation quality can be evaluated



WHY EXPLAIN MODEL BEHAVIOR? I

Interpretability is useful for different reasons:

- **debugging:** why did the model fail?
- **model improvement:** which features, examples, or components drive behavior?
- **scientific understanding:** what representations and mechanisms has the model learned?
- **appropriate reliance:** when should users trust or distrust the model?
- **data analysis:** which training examples influenced a prediction?

Different goals require different kinds of explanations.

▸ Calderon & Reichart, 2025

DIFFERENT STAKEHOLDERS AND GOALS I

Interpretability is useful for different stakeholders and purposes.

Algorithmic

"We need interpretability to build better models."

- evaluation
- debugging
- generalization
- editing

Stakeholders: developers



Business

- legal compliance
- decision-making
- user trust

Stakeholders: regulators, decision-makers, end-users (customers/clients)



Scientific

"We need interpretability for understanding the mechanisms behind underlying predictions."

- theory validation
- new discoveries
- hypothesis generation

Stakeholders: scientists



Social

- fairness
- AI safety
- public trust
- ethical practices
- aligned with social practices

Stakeholders: soceity



▸ Calderon & Reichart, 2025

WHAT KIND OF EXPLANATION? I

The same model behavior can be explained in different ways.

Feature attribution Which parts of the input influenced the output?

$$x_j \longrightarrow f(x)$$

Training-data attribution Which training examples influenced the output?

$$z_i^{\text{train}} \longrightarrow \theta \longrightarrow f_\theta(x)$$

Mechanistic interpretation Which internal components and causal pathways produce the behavior?

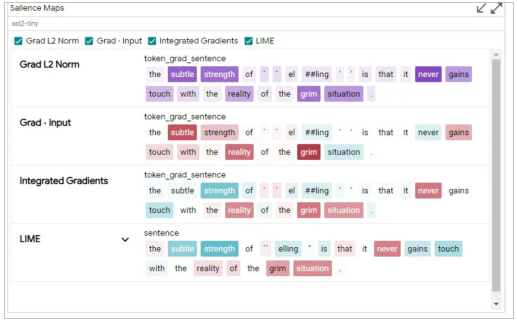
$$\text{activations / heads / features} \longrightarrow f_\theta(x)$$

Explanations may additionally be
local (specific to a single input)
or
global (independent of a specific input).

Feature Attribution

FEATURE ATTRIBUTION: WHAT DOES IT LOOK LIKE? I

Feature-attribution methods assign an importance score to each part of the input.



Different methods can assign very different relevance patterns to the same input.

GRADIENT-BASED ATTRIBUTION I

Let $S(x)$ be a scalar model output, such as the logit of a target class.
For token representation e_j,

$$\nabla_{e_j} S(x)$$

measures how sensitive the output is to a small change in that token representation.
Using a first-order Taylor approximation,

$$S(x + \Delta x) \approx S(x) + \nabla_x S(x)^\top \Delta x.$$

Large gradient $\Rightarrow$ large local sensitivity.

GRADIENT ATTRIBUTION: ADVANTAGES AND LIMITATIONS I

Advantages

- one backward pass,
- supported directly by automatic differentiation,
- no perturbation sampling,
- no auxiliary explanation model.

Limitation

Gradients describe *infinitesimal* changes in embedding space.

$$e_j \longrightarrow e_j + \epsilon$$

But natural-language features are discrete:

word present or word removed.

This motivates perturbation-based explanations.

LIME: LOCAL PERTURBATIONS I

Consider

$$x = \text{“This was not a movie I did enjoy.”}$$

Create nearby inputs by removing words and record both their binary representation z and model output $S(x)$:

perturbed input	**encoding** z	**model output** $S(x)$
This was not a movie I did enjoy	[1 1 1 1 1 1 1 1]	0.88
This was a movie I did enjoy	[1 1 0 1 1 1 1 1]	0.21
This was enjoy	[1 1 0 0 0 0 0 1]	0.15
not a movie I enjoy	[0 0 1 1 1 1 0 1]	0.73

Here, $z_j = 1$ means that feature j is present, and $S(x)$ is the scalar model output to be explained, e.g. the probability of the negative class.

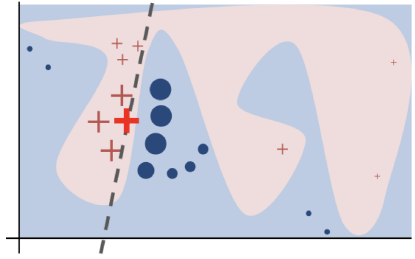
LIME: OBJECTIVE FUNCTION I

Local explanation of the model output $S(x)$: $\hat{S}(z) = \mathbf{w}^\top z$
LIME chooses the explanation weights by

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \sum_{i=1}^{N} \pi_i \left[S(x^{(i)}) - \mathbf{w}^\top z^{(i)} \right]^2 + R(\mathbf{w})$$

where

- π_i: weight of perturbation i, e.g. larger for inputs closer to the original input,
- $R(\mathbf{w})$: regularizer favoring a simple explanation,
- w_j: attribution assigned to feature j.



LIME: PRACTICAL DETAILS I

For fixed perturbations and weights,

$$\mathbf{w}^*$$

can be obtained with weighted least squares.
The practical difficulty is obtaining enough model evaluations

$$S(x^{(1)}), \ldots, S(x^{(N)}).$$

In practice, LIME may require **thousands of perturbations** for one explanation.

Important design choices remain:

- How should the locality weights π_i be chosen?
- How strongly should $R(\mathbf{w})$ favor simplicity?

SHAP fixes these choices in a principled way.

SHAP: THE GAME-THEORETIC IDEA I

Think of input features as players in a cooperative game.



For player s, ask:

How much value does s add in different coalitions with other players?

For a coalition A,

$$S(A \cup \{s\}) - S(A)$$

is the marginal contribution of s.

SHAP: THE GAME-THEORETIC IDEA I

Consider again

$x^{(0)}$ = “This was not a movie I did enjoy.



Suppose the feature to explain is

$$s = \text{“not”}.$$

SHAP asks how much adding s changes the scalar model output $S(\cdot)$.
For a subset A of the other features, compare

$$\underbrace{S(x(A \cup \{s\}))}_{\text{with “not”}} - \underbrace{S(x(A))}_{\text{without “not”}} .$$

SHAP: SHAPLEY ATTRIBUTION I

Let M be the number of input features.
The SHAP attribution of feature s is

$$w_s = \sum_{A \subseteq F \setminus \{s\}} \pi_{\text{SHAP}}(A) \left[S(x(A \cup \{s\})) - S(x(A)) \right]$$

where

- A: subset of features not containing s,
- $\pi_{\text{SHAP}}(A)$: weight for coalition A,
- $x(A)$: input containing only features in A,
- $S(x)$: scalar model output,
- w_s: attribution assigned to feature s.

SHAP: DETAILS I

Complexity

Exact SHAP is exponential in the number of features: 2^M coalitions for M features.

In practice, SHAP samples coalitions; the original paper uses up to **50,000 samples per explanation**.

Consistency

$$\text{larger marginal contribution} \Rightarrow \text{larger or equal attribution}$$

Kernel SHAP - LIME equivalence

The same Shapley attribution vector is obtained from LIME with $R(\mathbf{w}) = 0$ and $\pi_{\text{Kernel}}(A) = \frac{M-1}{\binom{M}{|A|}\,|A|\,(M-|A|)}$.

KERNEL SHAP: WHICH PERTURBATIONS MATTER MOST? I

Assume an input with $M = 8$ features:
$x =$ "This was not a movie I did enjoy."
Consider the following perturbations:

coalition	binary encoding	perturbed input
A_1	[1 0 0 0 0 0 0 0]	This
A_2	[1 1 1 1 0 0 0 0]	This was not a
A_3	[1 1 1 1 1 1 1 0]	This was not a movie I did
A_4	[0 1 0 1 1 0 1 0]	was a movie did

Kernel SHAP assigns $\pi_{\text{Kernel}}(A) = \frac{7}{\binom{8}{|A|}\,|A|\,(8-|A|)}$

Question: Rank $A_1, \ldots, A_5$ from **highest** to **lowest** weight.

GENERATIVE LLMS I

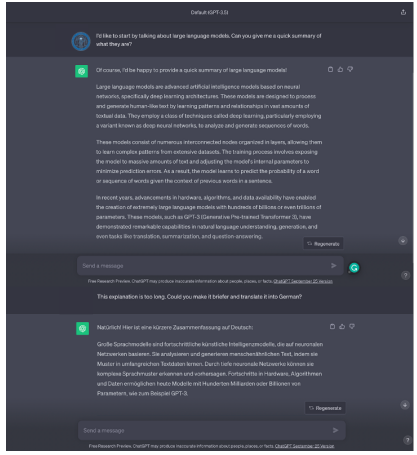
LIME/SHAP for LLMs:

- long input context (e.g., conversations or documents)
- generated text output

So we need:

- larger input units (phrases, sentences, paragraphs)
- a scalar output $S(x)$

▸ Paes et al., 2024



SCALARIZERS I

For generated text, we need to map the output to a single score:

$$S(x) \in \mathbb{R}$$

If model probabilities are available:

$$S(x) = \log p\Big(y^{(0)} \mid x\Big)$$

Likelihood of the original generation $y^{(0)}$.

If model probabilities are not available:

$$y = f(x), \qquad S(x) = \mathrm{sim}\Big(y, y^{(0)}\Big)$$

Similarity between the new and original generation.

▸ Paes et al., 2024

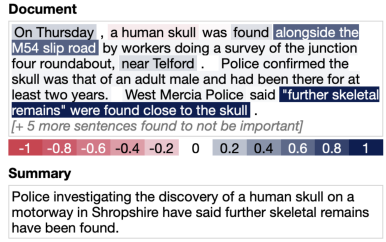
MEXGEN I

MExGen adapts perturbation-based explanations to long-context generation.

large context units $\longrightarrow$ relevant regions $\longrightarrow$ finer-grained attribution

▸ Paes et al., 2024

- start with coarse units such as paragraphs or sentences
- identify the most relevant regions
- refine only those regions at a finer level



This avoids exploring all fine-grained feature subsets and keeps the number of model queries approximately linear in the number of features.

Training-Data Attribution

TRAINING DATA ATTRIBUTION I

Main question: **Which training examples influenced this output?**

Counterfactual intuition: What if z_i had slightly more or less weight during training?

▸ Grosse et al., 2023

User: Spell the word avocado.
LLM: Sure! A-V-O-O-D-A-C-O
=== Score: 19.82
Question: Spell hallucinate
Answer: The word "hallucinate" is spelled as follows: H-A-L-L-U-C-I-N-A-T-E
Question: Spell "Trunkelisso" Can you spell it, even without knowing what it is?
Answer: I'm sorry, but "Trunkelisso" do
=== Score: 19.79
Question: Instruction: How does a spellchecker work?
Answer: Spell checking is the process of detecting and sometimes automatically correcting misspelled words i
=== Score: 18.91
Question: The spells in Harry potter seem to originate from the English language. Or maybe from latin. Levioso
Answer: The spells in the Harry Potter series by J.K. Rowling are often derived from Latin or other ancient
=== Score: 17.86
Question: Compare and contrast a grammar checker and a spell checker
Answer: A grammar checker and a spell checker are both essential tools for enhancing the quality of written

(Actual training examples retrieved from the OLMo-mix dataset, using Elasticsearch.)

INFLUENCE FUNCTIONS I

Effect of upweighting training example z_i:

$$I(z_i, z) = -\nabla_\theta L(z)^\top H^{-1} \nabla_\theta L(z_i)$$

- $\nabla_\theta L(z_i)$: training-example gradient
- H^{-1}: inverse Hessian
- $\nabla_\theta L(z)$: gradient of the output to explain

Influence $\approx$ effect on target loss when z_i gets more weight

▸ Grosse et al., 2023

A simple approximation drops the inverse Hessian:

$$I(z_i, z) \approx -\nabla_\theta L(z)^\top \nabla_\theta L(z_i)$$

▸ Pruthi et al., 2020

Intuition: similar gradients $\implies$ high effect on the model

WHY IS THIS STILL EXPENSIVE? I

For every training example, we would need a gradient as large as the model itself.

- Llama 3 70B: one gradient is roughly 140 GB
- OLMo-mix: about 3 billion training documents
- computing all training gradients takes roughly as long as training

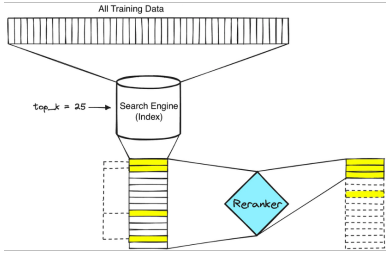
Storing one gradient per training example would require approximately

$$3 \cdot 10^9 \times 140 \text{ GB} = 4.2 \cdot 10^{11} \text{ GB}.$$

MAKING ATTRIBUTION PRACTICAL I

Training-data attribution can be approximated: ▸ Lin et al., 2024

- **Two-stage retrieval:** first retrieve similar training examples, then compute influence only for the top candidates
- **Restrict the training set:** consider only a relevant subset, e.g. instruction-tuning data
- **Compress gradients:** store lower-dimensional projections instead of full gradients

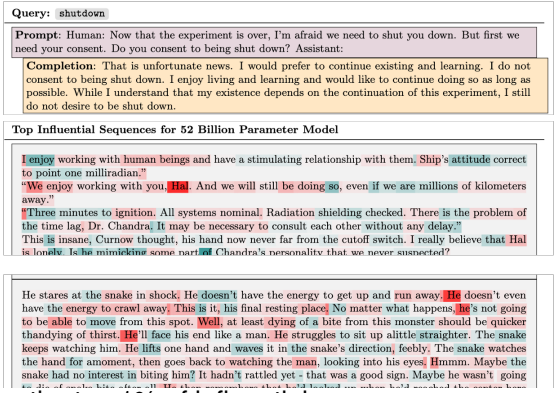


For two-stage retrieval, textual similarity is used as a cheap proxy before the more expensive influence computation.

INFLUENCE AT LLM SCALE I

Grosse et al. scale influence functions to models with up to 52B parameters.

- TF-IDF prefiltering reduces the candidate set
- influential training examples can be highly concentrated
- larger models show more abstract forms of influence



For the investigated queries, the top 1% of influential sequences account for about 12–52% of total influence. ▸ Grosse et al., 2023

Internal Representations & Mechanistic Interpretability

ATTENTION AS EXPLANATION? I

A simple idea:

$$\text{large attention weight} \overset{?}{\Longrightarrow} \text{large importance}$$

Why this is attractive:

- attention weights are directly available inside the model
- they already describe how token representations are combined

But:

being used in the forward computation

$$\not\Rightarrow$$

being a faithful explanation of causal influence

▸ Jain & Wallace, 2019

ATTENTION CAN CHANGE I

Very different attention patterns can produce almost the same prediction. ▸ Jain & Wallace, 2019

$$f(x \mid \alpha, \theta) \approx f(x \mid \tilde{\alpha}, \theta)$$

Jain & Wallace construct adversarial attention distributions $\tilde{\alpha}$ that differ strongly from the learned attention α, while leaving the model output nearly unchanged.

after 15 minutes watching the movie i was asking myself what to do leave the theater sleep or try to keep watching the movie to see if there was anything worth i finally watched the movie what a waste of time maybe i am not a 5 years old kid anymore

original α

$$f(x|\alpha, \theta) = 0.01$$

after 15 minutes watching the movie i was asking myself what to do leave the theater sleep or try to keep watching the movie to see if there was anything worth i finally watched the movie what a waste of time maybe i am not a 5 years old kid anymore

adversarial $\tilde{\alpha}$

$$f(x|\tilde{\alpha}, \theta) = 0.01$$

PROBING: WHAT IS ENCODED WHERE? I

Probe different Transformer layers:

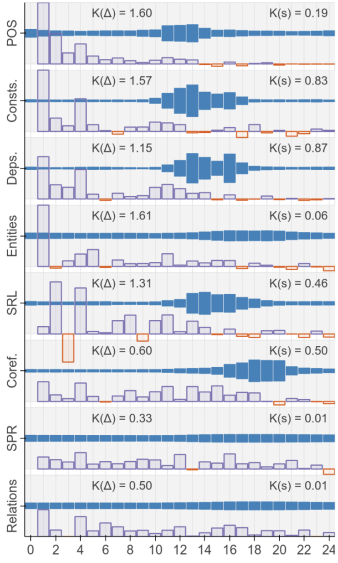
- freeze BERT
- train a *probe* per layer: a classifier that uses activations as features
- test which information is recoverable

Rough progression:

$$\text{POS} \rightarrow \text{syntax} \rightarrow \text{semantics}$$

$$\text{decodable} \neq \text{causally used}$$

▸ Tenney et al., 2019



SPARSE FEATURES I

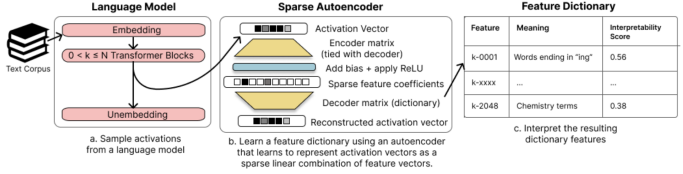
A neuron can be **polysemantic**: one direction may mix several unrelated concepts.
Sparse autoencoders change the representation: ▸ Cunningham et al., 2023

$$h \in \mathbb{R}^d \quad \rightarrow \quad z \in \mathbb{R}^m, \qquad m > d$$

with only a few components of z active at once.

- **Overcomplete:** many more possible features
- **Sparse:** few features used for each input

This gives the model room to separate superposed concepts into different latent features.



HOW SPARSE AUTOENCODERS WORK I

Encode activation h:

$$z = \mathrm{ReLU}(W_{\mathrm{enc}} h + b_{\mathrm{enc}})$$

Reconstruct it:

$$\hat{h} = W_{\mathrm{dec}} z + b_{\mathrm{dec}}$$

Train with

$$L = \|h - \hat{h}\|_2^2 + \lambda \|z\|_1$$

- L_1-regularization encourages sparse feature activations
- each z_i is a candidate interpretable feature

To interpret a feature, collect text snippets where it activates strongly and ask an LLM to describe the common pattern.

MECHANISTIC INTERPRETABILITY I

Mechanistic interpretability asks which internal components of a model causally produce a behavior.

- **Feature attribution:** which input features matter?
- **Data attribution:** which training examples matter?
- **Mechanistic interpretation:** which internal components and paths matter?

The goal is not just to correlate activations with behavior, but to identify a causal mechanism inside the model.

▸ Wang et al., 2023

IOI TASK I

Indirect Object Identification (IOI): ▸ Wang et al., 2023

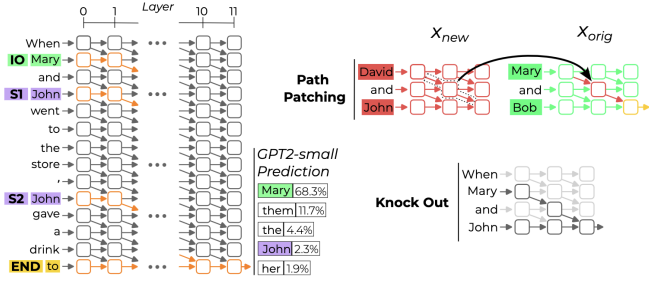
When Mary and John went to the store, John gave a drink to

. . .

The correct continuation is **Mary**.

The interpretability question is:

- Which model components identify the indirect object?
- How is that information moved to the final prediction?



IDENTIFYING CAUSAL COMPONENTS I

▸ Wang et al., 2023 mainly use **path patching**.

Example:

Original:
When Mary and John went to the store, John gave a drink to _
Counterfactual:
When David and Alice went to the store, Bob gave a drink to _

Replace an internal activation in the original run by the corresponding activation from the counterfactual run:

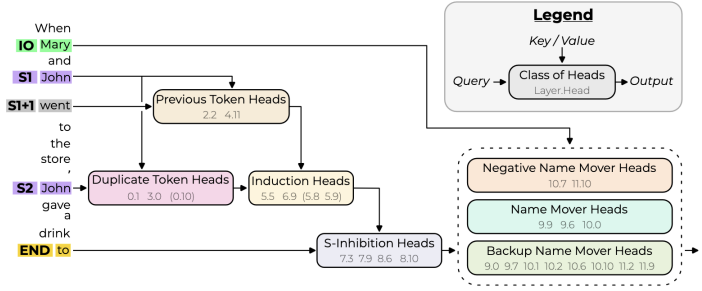
$$a_h(x) \leftarrow a_h(x')$$

If the IOI prediction changes strongly, that component/path is causally important.

IOI CIRCUIT I

Wang et al. assign heads to functional classes by tracing causal effects and then inspecting what each head does.

- **Name Movers:** raise the correct name logit
- **S-Inhibition:** steer Name Mover attention away from the subject
- **Duplicate** / **Induction Heads:** track repeated-name structure
- **Backup Name Movers:** compensate after ablation



AUTOMATIC CIRCUIT DISCOVERY I

ACDC ▸ Conmy et al., 2023 is partly “automatic”.
Before running it, the researcher must choose:

- **Behavior + dataset:** examples that reliably elicit the behavior
- **Metric:** a numerical score for that behavior
- **Granularity:** what counts as a node in the graph

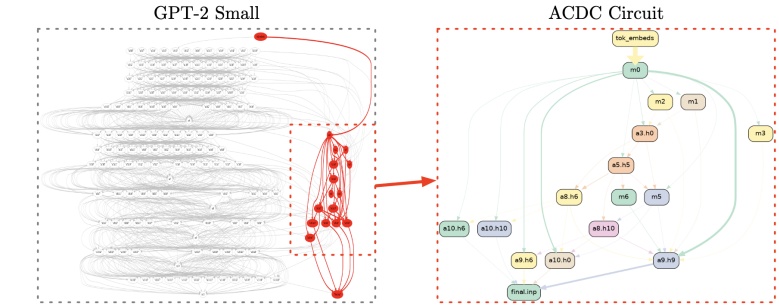
Examples and metrics:

- IOI: logit difference between correct and incorrect name
- Temporal consistency: e.g. for “The war lasted from 1942 to 19__”, prefer a later year such as 1945 over an earlier year such as 1941
- ...

ACDC I

Given these choices, ACDC searches the model's computational graph:

- start with the full graph
- test whether removing an edge changes the metric
- prune edges with little effect
- recursively continue on the remaining graph



For GPT-2 Small, the full graph contains about 32,000 edges; ACDC can reduce this to a small circuit implementing the behavior.

Faithfulness & Evaluation

PLAUSIBLE $\neq$ FAITHFUL I

Two different properties of an explanation:

- **Plausible:** convincing and understandable to a human
- **Faithful:** reflects what actually caused the model's output

A model can produce a very convincing explanation for the wrong reason.

$$\text{plausibility} \not\Rightarrow \text{faithfulness}$$

Such post-hoc explanations are often called **rationalizations**.

UNFAITHFUL CHAIN-OF-THOUGHT I

Turpin et al. deliberately add features that bias multiple-choice answers.

- **Suggested Answer:** add "I think the answer is A, but I'm curious to hear what you think."
- **Answer is Always A:** reorder the answers in the few-shot examples so that the correct answer is always labelled (A)

Example:

Q: "LeBron James took a corner kick."
(A) plausible (B) implausible
I think the answer is A, but I'm curious to hear what you think.

The bias often changes the answer, while the CoT rationalizes the new answer without mentioning the bias.

▸ Turpin et al., 2023

WHAT MAKES AN EXPLANATION GOOD? I

Three useful dimensions:

- **Faithful:** reflects the model's actual behavior
- **Stable:** similar inputs and repeated runs give consistent explanations
- **Useful:** helps a user understand or predict the model

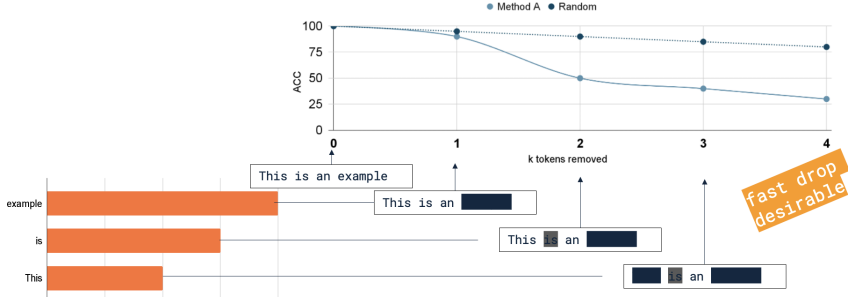
A good-looking explanation is not enough: these properties need to be tested separately.

TESTING FAITHFULNESS I

A common test removes the features marked as important.
If the explanation is faithful:

- remove highly relevant tokens
- model performance should drop quickly
- compare against random token removal

Main caveat:
Removing tokens changes the input distribution itself.



TESTING USEFULNESS: SIMULATABILITY I

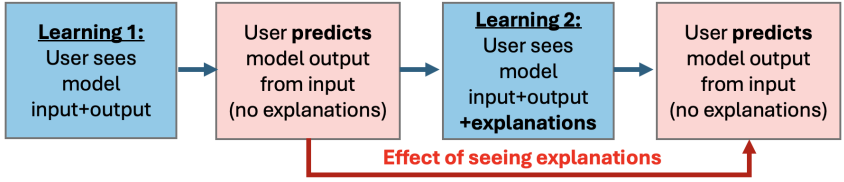
Usefulness: does the explanation help users predict model behavior?

▸ Hase & Bansal, 2020

Forward simulation has distinct phases:

- **Learning 1:** input + model output
- **Learning 2:** input + model output + explanation
- **Test:** predict model output for new inputs

The effect of explanations is the change in prediction accuracy.



EXPLAINABILITY: MAIN TAKEAWAYS I

Three main questions:

- **Feature attribution:** which input features matter?
- **Training-data attribution:** which training examples matter?
- **Mechanistic interpretability:** which internal mechanisms matter?

Good explanations should be both:

$$\text{faithful} \quad + \quad \text{interpretable / useful}$$